

Received May 18, 2021, accepted July 1, 2021, date of publication July 12, 2021, date of current version July 21, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3096609

Graph Convolutional Embeddings for Recommender Systems

PAULA G. DURAN¹, ALEXANDROS KARATZOGLOU², JORDI VITRIÀ¹, XIN XIN³,
AND IOANNIS ARAPAKIS⁴

¹Departament de Matemàtiques i Informàtica, Universitat de Barcelona, 08007 Barcelona, Spain

²Google Research, London N1C 4AG, U.K.

³School of Computing Science, University of Glasgow, Glasgow G12 8QQ, U.K.

⁴Telefonica Research, 08019 Barcelona, Spain

Corresponding author: Paula G. Duran (paulagomezduran@gmail.com and paula.gomez@ub.edu)

This work was supported in part by the Ministerio de Asuntos Económicos y Transformación Digital (MINECO)/Fondo Europeo de Desarrollo Regional (FEDER), Unión Europea (UE), under Project RTI2018-095232-B-C21, and in part by the Generalitat de Catalunya under Project 2017SGR1742.

ABSTRACT Modern recommender systems (RS) work by processing a number of signals that can be inferred from large sets of user-item interaction data. The main signal to analyze stems from the raw matrix that represents interactions. However, we can increase the performance of RS by considering other kinds of signals like the context of interactions, which could be, for example, the time or date of the interaction, the user location, or sequential data corresponding to the historical interactions of the user with the system. These complex, context-based interaction signals are characterized by a rich relational structure that can be represented by a multi-partite graph. Graph Convolutional Networks (GCNs) have been used successfully in collaborative filtering with simple user-item interaction data. In this work, we generalize the use of GCNs for N-partite graphs by considering N multiple context dimensions and propose a simple way for their seamless integration in modern deep learning RS architectures. More specifically, we define a graph convolutional embedding layer for N-partite graphs that processes user-item-context interactions and constructs node embeddings by leveraging their relational structure. Experiments on several datasets show the benefits of the introduced GCN embedding layer by measuring the performance of different context-enriched tasks.

INDEX TERMS Graph convolutional network, interaction context, embedding layer, factorization machines, neural networks, collaborative filtering, context-aware recommendation.

I. INTRODUCTION

The amount of information available on the internet and beyond far exceeds any human capacity to take account of it, making it particularly challenging for the average user to locate items of interest or relevance online. Recommender Systems, and in particular collaborative filtering methods, have helped alleviate the information overload problem and have become one of the main ways people explore, filter and discover content and information.

One of the undertakings when building an online RS is to use all available data to generate recommendations that are useful and relevant. Data that is collected from online systems on user interactions can include a broader range of information than just the indicators of user-item interactions, such as the time and day of the interaction, the location of the

user, the type of the device that acted as the medium for the interaction, the historical interactions of the user during and before that session, as well as the past recommendations the user received during her interactions with the system. This type of meta-data is commonly known as *context* [1].

User, item, and context data are often collected in the form of user-id, item-id and context-id's. These data can be represented in a N-dimensional sparse tensor or, equivalently, as an *N-partite graph*, where users, items, and context dimensions define different types of nodes that are connected when there are interactions that involve them. In Figure 1, we show the case of a given user that has interacted with an item under several contexts.

Several methods have been proposed for processing such data. Some of the methods rely on factorizing the user-item-context interactions [23], [28], [31], [33], while other more recent approaches use deep neural networks to produce powerful representations of the user-item-context

The associate editor coordinating the review of this manuscript and approving it for publication was Pasquale De Meo.

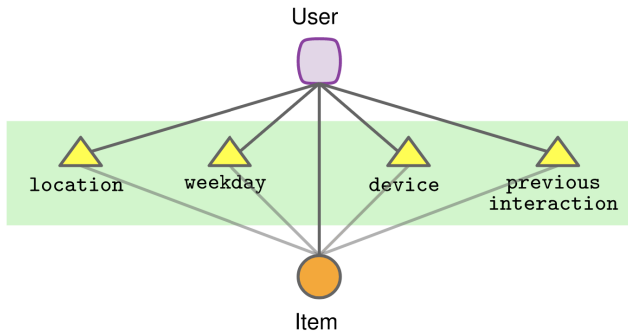


FIGURE 1. Illustration of a sub-graph from the interaction of a user which interacted with an item under four different context taken into account: weekday, location, device, previous interaction.

interaction data [15], [18], [20], [22]. Typically, Deep Learning (DL) methods compute embeddings at the first layers of the network, which typically consist of standard dense layers [4].

Embedding vectors, or latent factors, computed by modern collaborative filtering techniques like factorization machines or deep factorization machines, rely on a loss function to provide a signal (gradient) about the existence of a user-item interaction. In most of the cases, recommendation is based on pair-wise products between different embedding vectors. The models used to learn these latent representations are mainly composed of fully connected layers that do not present any interesting inductive bias within.

Graph Convolutional Networks [26] have recently gained prominence with applications in several domains, from node classification [26] to physical systems simulation [35]. Learning dense and low-dimensional node representations from graph-structured data has become the keystone in many practical application scenarios, including typical user-item collaborative filtering [2], [7]. Their main advantage is their ability to deal with node information and relational structure at the same time. GCNs learn node representations as the summary of the node's neighborhood, which allows them to effectively encode the graph's (higher-order) topological structure and connectivity information [9], [21], [26]. Moreover, one of the advantages of GCN is that they seamlessly incorporate graph structure information in the form of the graph adjacency matrix into the model. Hence GCNs do not need to learn that structure and can focus their modeling capacity on learning good node representations.

Since many types of data come in the form of graphs, or can be converted to graphs, GCNs have proven to be an extremely useful addition to the existing DL modeling approaches. GCNs have been proposed for collaborative filtering algorithms in particular types of recommendation tasks, such as social recommendations [14], item-to-item recommendations [44], or rating prediction using user-item ratings [7]. While these GCN-based recommendation methods demonstrate the potential of GCNs in RS, they are built to address very specific recommendation settings and cannot be easily extended to other architectures.

In this work, we explore how using relational inductive biases within deep learning architectures can facilitate learning about entities in a collaborative context-aware recommendation scenario. To this end, we generalize the use of GCNs in the area of context-aware RS by building a GCN layer that can be used as substitute of a classical embedding layer, allowing its use in practically any recommendation model. This layer can process both implicit and explicit feedback for generalized user-item-context interactions, while also allowing for seamless integration of side information. We refer to the proposed layer as Graph Convolutional Embedding (GCE) layer hereinafter.

By applying GCE layers to existing collaborative filtering models, we significantly improve their performance in terms of off-line ranking metrics on several datasets and models. We also report several experiments to understand the causes of this improvement, being the most important conclusion that the use of a GCE, which endows the model with a strong relational bias during the training process, is sufficient to capture a great deal of interactions statistics prior to any learning.

II. RELATED WORK

Several methods have been proposed to deal with context-aware recommendation data. In what follows, we review some related previous research efforts from the perspective of context-aware recommendation and graph convolutions.

A. CONTEXT-AWARE RECOMMENDATION

The traditional solution for collaborative filtering is Matrix Factorization (MF) [27], even though it presents some clear limitations to model user-item interactions, such as not being able to incorporate side-features, not dealing properly with sparse data or the cold-start problem [31].

Factorization Machines (FM) [31], [33] generalize MF by making use of inner product to model pairwise latent representations of feature interactions and then formulating their sum to get the predicted scores. They also allow to add side-information of the features, and so FM had demonstrable success due to its high performance and low (linear) computational complexity.

Several efforts have also been made to enhance FM with DL components [15], [20], [29], [43]. For example, Neural Factorization Machines (NFM) [18] leverages the flexibility of neural networks to replace dot products of matrix factorization and to learn higher-order signals. Deep FM [15] resembles the original FM and it adds a term of concatenated embeddings. Finally, Neural Collaborative Filtering [20] borrows ideas from Deep FM, thus combining the FM term with input features instead of their embeddings.

In order to extend the application of MF to context-aware data, Karatzoglou *et al.* [23] present an N -dimensional tensor factorization method that accounts for context features along with the user-item interactions. In this modelling approach, user-item-context data are represented as tensors $\mathbf{X} \in \mathbb{R}^N$, where $N = m \times n \times c_1 \times c_2 \dots c_{N-2}$, being

$m, n, c_1, \dots, c_{N-2}$ the dimensions of the user, item and different contexts variables respectively. Typically, $\mathbf{X}$ contains non-zero entries at the positions corresponding to a user-item-context interaction and zero (or missing values) otherwise, hence this tensor is extremely sparse. The intuition behind using tensor factorization to solve this problem is that, within a certain context, there are latent features that determine the likelihood of a user-item interaction. In the case of FM models, context can be naturally included in FM models as new features. This results in a model that, instead of using a tensor inner product as in [23], considers the sum of pairwise operations among features.

B. GRAPH CONVOLUTIONAL NETWORKS

All methods reported in the previous section [18], [20], [27], [31] treat input features (user, item or context) as independent elements that are not explicitly related in any way. Thus, they solely rely on the signals of the user-item interactions to capture their latent representations (embeddings). In order to capture embeddings that account for the different relational structures, we propose the use of Graph Convolutional Networks, which allow to capture structure-sensitive latent representations.

Graph Convolutional Networks [9], [11], [13], [26] are a class of neural networks that was introduced for modeling graph-structured data. The first generation of graph convolution models was motivated from the spectral perspective [9], [11], [21]. Based on this body of research, Kipf and Welling [26] proposed a new model, known as GCNs, based on the re-normalization trick on the spectral filter and also on graph convolutions for semi-supervised node classification. Due to the success of GCN, both in terms of performance and efficiency, there has been an surge of research on the topic. Berg *et al.* [7] proposed to utilize GCN to perform matrix completion on user ratings. Hamilton *et al.* [16] and Chen *et al.* [10] proposed sampling-based graph convolution. Wu *et al.* [42] treated the neighborhood aggregation as a pre-computing process and accelerated the training of GCN. Veličković *et al.* [37] proposed to use the attention mechanism, other than topology (degree) information, to learn the node importance when constructing the convolution kernel.

While different versions of GCNs have been proposed in the literature, they all share some basic principles. The assumption of GCNs is that the embedding of a node must result from the aggregation of itself and its neighbors. More specifically, GCNs aim to learn a function of node features on graph topological data. It is defined as a function over the graph $\mathcal{G} = (\mathcal{V}; \mathcal{E})$, where $\mathcal{V}$ denotes the vertices or nodes of the graph and $\mathcal{E}$ is the edges, which takes the following inputs:

- An original feature matrix $Z \in \mathbb{R}^{|\mathcal{V}| \times d}$, where d is the dimension of the original features. Usually, Z is sparse and uses one/multi-hot encoding to represent features.
- A representative description of the graph structure in matrix form, which is called adjacency matrix $A \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$. a_{ij} is the (i, j) -th entry of A with $a_{ij} = 1$

denotes there is an edge between node i and node j . Otherwise, $a_{ij} = 0$.

and produces the output, which is a $|\mathcal{V}| \times f$ embedding matrix with f being the output embedding size per node. GCNs can thus be seen as a processing layer in deep networks that takes into account the topological structure of the input graph. Moreover, it is also possible to stack multiple GCN layers.

The layer-wise propagation rule of multi-layer GCN can be formulated as $H^{(l+1)} = \sigma(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)})$ where σ is the activation function, $H^{(l)}$ denotes the hidden representation after l -th layer of graph convolution, $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is the trainable weight matrix with d_l being the hidden size of the l -th layer¹; $\hat{A} = A + I_N$, where I_N is an identity matrix, is the adjacency matrix of the graph $\mathcal{G}$, with added self-connections in order to include the node's own features; and $\hat{D} = D + I_N$, where D is the diagonal degree matrix: $D_{ii} = \sum_j A_{ij}$. The normalized adjacent matrix $\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}}$ serves as the keystone to perform neighborhood aggregation between nodes of the graph.

C. GCN IN RECOMMENDER SYSTEMS

GCN were first introduced to recommendation problems in Graph Convolutional Matrix Completion (GC-MC) work [7], which aimed to perform rating prediction based on user-item interactions.

GC-MC consists of a graph-convolutional auto-encoder framework which is based on message passing through different kinds of edges (ratings). The rating matrix $R \in \mathbb{R}^{m \times n}$ is used to construct the adjacency matrix of a bipartite graph between user and item nodes. The edges in this graph correspond to the ratings the user have given to the item. Different ratings are captured by different kinds of edges. The learning task is predicting the value of an unobserved entry in R and, therefore, can be seen as a link prediction problem.

In GC-MC, the latent user and item node representations are based on differentiable message passing on the bipartite interaction graph.² The graph convolutional layer performs local operations that take the direct neighbors of a node into account, whereby the same transformation is applied across all locations in the graph. A specific transformation is assigned for each rating level, resulting in edge-type specific messages $\mu_{j \rightarrow i, r}$ from items j to users i according to rating r . The equation is then defined in the following form: $\mu_{j \rightarrow i, r} = \frac{1}{s_{ij}} W_r z_j$ where s_{ij} is a normalization coefficient that can be either $|\mathcal{N}_i|$ (left normalization) or $\sqrt{|\mathcal{N}_i| |\mathcal{N}_j|}$ (symmetric normalization), with $\mathcal{N}_i$ denoting the set of neighbors of node i ; W_r is the trainable weight matrix regarding to the rating r and z_j is the feature vector of node j .

In [44], GCNs are used for RS by processing item representations generated by both co-clicking patterns and

¹Note that the initial layer of H would be $H^{(0)} = Z$ and $d_0 = d$. In the final L layer $d_L = f$.

²These latent user and item representations are used to reconstruct the rating links through a bi-linear decoder

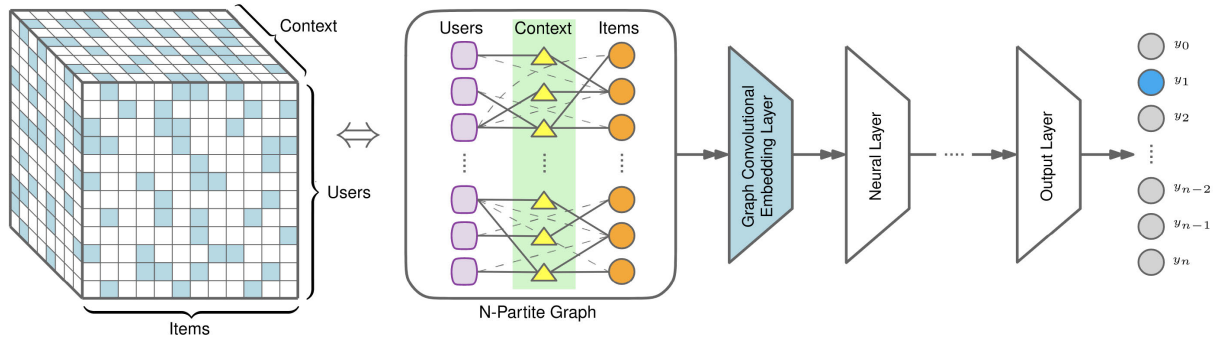


FIGURE 2. User-item-context interaction tensor $\mathbf{X}$ is equivalent to a three-partite graph $\mathcal{G}$ under the setting of a single context dimension. Edges correspond to interaction events. $\mathcal{G}$ is the input to the GCE layer that generates the node representations that are then used by layers of a recommendation model.

content-based similarity data. The item similarity graph is processed with graph convolutions to generate recommendations based on the items the users have interacted with. Note that this work considers only the item-similarity graph and not the interactions between users, items and context. Additional methods using GCNs for collaborative filtering have also been developed, such as those proposed by Wang *et al.* [41], Wang *et al.* [40], and Sun *et al.* [36].

However, these methods focus on bipartite user-item interactions, while our work aims to generate recommendations based on more generalized user-item-context interactions. Thus, in order to use GCN in our work, we will need to extend message passing to multiple dimensions and, then, integrate it in those variations of FM models that naturally allow multiple feature dimensions.

III. GRAPH CONVOLUTIONAL EMBEDDINGS

In this work, we propose a general Graph Convolutional Embedding layer that can be used as a building block in deep neural recommendation methods. We define a graph convolution layer that can model complex user-item-context interactions with an arbitrary number of different contexts. User-item-context data can be represented in a N -dimensional tensor $\mathbf{X} \in \mathbb{R}^{m \times n \times c_1 \times c_2 \times \dots \times c_{N-2}}$, as described in subsection II-A. These interactions can be modeled as a N -partite graph, where users, items, and context dimensions are different types of nodes that are connected to represent a user-item interaction under a particular context. We show this relationship (for a single context dimension) in Figure 2, the user-item-context data is depicted both as a tensor and an equivalent N -partite graph.

Graph convolutional embeddings use N -partite graphs as input to generate representations for the user, item, and context nodes, which can then be used in downstream recommendation models. Note that, in our model, the existence of context is not necessary and, also, that GCE works both with implicit and explicit (rating) feedback data, as this only depends on the loss function of the downstream recommendation task.

A. MODEL

We describe the GCE layer and the node convolutions over the N -partite graph. In the GC-MC model [7], the local convolution is seen as a form of message passing from the representations of item j to the representation of user i , and vice-versa. We use a different message passing formulation in order to generalize it to context, which can have either one or several dimensions (from c_1 to c_{N-2}). Here, the messages to each node pass not only from the user (or item) nodes but also from the context nodes that correspond to the user-item interaction. That is, when updating a user node i , the update includes messages not only from the items that the user interacted with (the item neighborhood of the user) but also from the context nodes that have edges with both the user and the item of the update.

The update rule can be formulated as³:

$$\mu_{j,e,\dots,o \rightarrow i} = \frac{1}{s_{ji}} W_u h_j + \frac{1}{s_{ei}} W_{c_1} h_e + \dots + \frac{1}{s_{oi}} W_{c_{N-2}} h_o \quad (1)$$

where h_j is the input embedding for node j . In the first layer, $h_j = z_j$, index j is used to index nodes that correspond to users, i for item nodes, and e for context c_1 , etc. W_u is the trainable weight matrix for the user factors, W_{c_1} for the context c_1 , etc. s_{ji} is the normalization factor $s_{ji} = \sqrt{|\mathcal{N}_j + 1| |\mathcal{N}_i + 1|}$, where $\mathcal{N}_j$ denotes the degree of node j .

Apart from the messages coming from neighbors, the node should also keep its own features. Finally, we apply an element-wise activation function to add non-linearity to our model. As a result, the new embedding for node i is formulated as:

$$\begin{aligned} h_i^{(1)} &= \sigma \left(\frac{1}{\mathcal{N}_i + 1} W_i h_i + \mu_{j,e,\dots,o \rightarrow i} \right) \\ &= \sigma \left(\frac{1}{\mathcal{N}_i + 1} W_i h_i + \frac{1}{s_{ji}} W_u h_j + \dots + \frac{1}{s_{oi}} W_{c_{N-2}} h_o \right) \end{aligned} \quad (2)$$

³Here the formulation is used for implicit feedback data. For explicit rating data, we just need to add the subscript of rating r to the trainable weight matrices (e.g., $W_u \rightarrow W_{u,r}$).

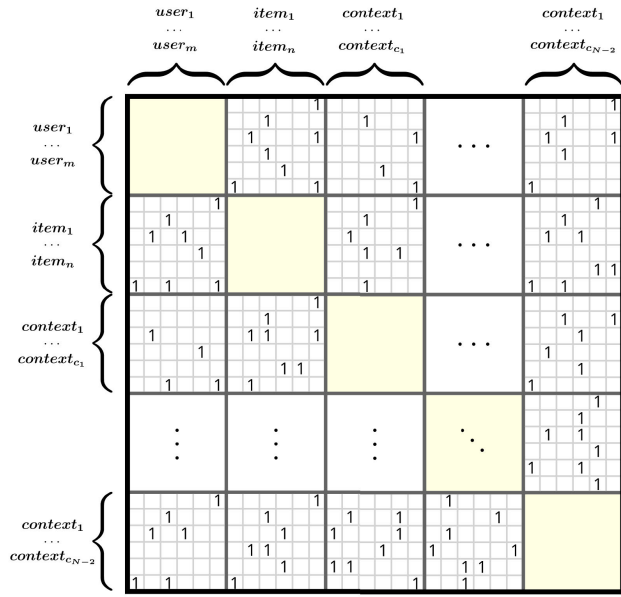


FIGURE 3. Schema providing an intuition of how matrix A is built.

where the first term denotes the preserved features of node i itself. If we use a shared weight matrix (i.e., $W_i = W_u = \dots = W_{c_{N-2}} = W$),⁴ Equation 2 can be written into a more concise form:

$$h_i^{(1)} = \sigma\left(\left(\frac{1}{N_i + 1}h_i + \frac{1}{s_{ji}}h_j + \dots + \frac{1}{s_{oi}}h_o\right)W\right) \quad (3)$$

We can use H to denote the whole input matrix, then Equation 3 can be rewritten into the matrix formulation:

$$H^{(1)} = \sigma(\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}HW) \quad (4)$$

where $H^{(1)}$ is the new embedding matrix after graph convolution. $\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$ is the normalized adjacent matrix defined on the N -partite interaction graph (as shown in Figure 2). It conforms with the original definition of GCN [26], as discussed in subsection 2.2.

1) GRAPH ADJACENCY MATRIX CONSTRUCTION

We construct the adjacency matrix A of the N -partite graph based on the user-item-context interaction data. For every interaction, we will have a multi-field categorical vector which contains the indexes of the $\langle \text{userID}, \text{itemID}, \text{contextID}_1, \text{contextID}_2, \dots, \text{contextID}_{N-2} \rangle$ of the interaction. Note that we re-index all the nodes in order to provide a unique identifier for all. As a result, the total number of nodes in the constructed graph is $m + n + c_1 + \dots + c_{N-2}$ (i.e., $|\mathcal{V}| = m + n + c_1 + \dots + c_{N-2}$).

An illustration of the adjacency matrix construction is shown in Figure 3. Note that A is built just with the training set, and naturally has a block-diagonal structure. Note that user-items, user-context and item-context are connected with bidirectional links while user-user, item-item and

context-context nodes are not linked, indicating that context can influence the user preference or how an item is perceived.

B. GCE FOR FACTORIZATION MACHINES

In this section we describe the use of GCE with FM [33], namely GCE-FM. The GCE layer can be essentially used with any recommendation model that uses and computes embeddings through gradient descent, such as MF [3], NFM [18], NCF [20], etc. (see subsubsection IV-A1 for a detailed description of some of those models).

The combination of GCE with the FM model is relatively straightforward. While the standard FM models pairwise interactions between different context variables through inner product, the result is linear to the embedding factors. When combined with GCE, the result remains linear with respect to the output of the GCE layer. Note that the GCE layer is fully differentiable, hence the model parameters can be easily computed with back-propagation and stochastic gradient descent. The embeddings computed by the GCE layer already capture high-order interaction signals between the different nodes of the user-item-context graph; hence FM, in effect, acts as an additional layer of interactions computation.

An entry (interaction record) in the original input tensor $\mathbf{X}$ can be represented as a sparse categorical vector $x \in \mathbb{R}^{|\mathcal{V}|}$. Then, the predicted score based on the input x can be formulated as:

$$\hat{y}(x) = w_0 + \sum_{p=1}^{|\mathcal{V}|} w_p x_p + \sum_{p=1}^{|\mathcal{V}|} \sum_{q=p+1}^{|\mathcal{V}|} x_p x_q \langle \mathbf{g}(x_p), \mathbf{g}(x_q) \rangle \quad (5)$$

where x_p denotes the p -th element in x (corresponding to a context variable p), w_0 is the global bias, w_p models the bias of p , $\mathbf{g}(x_p)$ is the output GCE embedding corresponding to the context variable p , and $\langle \cdot \rangle$ denotes inner product of two vectors. From this perspective, the original FM would be a special case of GCE-FM, when $\mathbf{g}(x_p)$ is simplified to an embedding table lookup operation without graph convolution. Thus, for any other RS-models we can compute its GCE-version like: GCE-NFM, GCE-WD, GCE-DFM... by just changing the way of capturing their latent representations.

C. TRAINING

After combining GCE with the downstream recommendation model, we train the whole parameters in an end-to-end fashion. For top- K recommendation, we use the BPR loss [32] as the training function:

$$L = - \sum \log(\sigma((S(u; i; c) - S(u; j; c)))) \quad (6)$$

where $S(u; i; c)$ is the score prediction of a positive interaction between user u and an item i in a particular context c , and $S(u; j; c)$ is the score prediction of the corresponding negative sample (same u and c but sampled item j denoting non-interaction between u - j in a given c).

⁴It is a common practical use case to prevent overfitting.

D. TIME COMPLEXITY

Since GCE can be seamlessly combined with downstream recommendation models, we only need to consider the additional cost introduced by the graph convolution operation. Consider a GCE layer as shown in Equation 4. As the normalized adjacent matrix can be pre-computed, the additional complexity only comes from two matrix multiplication operations.

The first multiplication is between the normalized adjacent matrix (i.e., $\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}$) and the input matrix. However, due to the sparse structure of the adjacent matrix, the complexity is only related to the number of non-zero entries in the matrix. The second matrix multiplication operation is a standard matrix multiplication with a complexity pseudo-polynomial (linear) to the number of nodes $|V|$. As a result, the total computational cost of a GCE layer is linear with respect to the number of nodes and edges of the constructed graph $O(|V||E|d^2)$.

IV. EXPERIMENTS

In this section we answer several experimental questions:

- 1) Does the performance of state-of-the-art context-aware recommendation models increase in a consistent way when using GCE?
- 2) What part of the performance increment is due to the relational inductive bias that is encoded within GCE?
- 3) How can we measure the utility of different relational structures for recommending items?

First, we evaluate the proposed GCE by combining it with three state-of-the-art recommendation models, which can be trained with back-propagation and which can be naturally extended to do context-aware recommendations. We compare the performance of every model in two different situations: 1) by using a GCE layer and 2) by using a traditional embedding layer. We do so by training all models end-to-end on the same dataset, and also using the same loss function and criteria to decide all meta-parameters.

Additionally, we evaluate the case of incorporating side-information (if the dataset provides it) in some of the nodes in order to show how GCE can easily incorporate any kind of information, either as nodes or as information inside each node. We evaluate for each dataset whether the GCE layer helps to improve the performance of the model or not. For example, taking FM as a baseline model we will compare the three following models, all trained end-to-end:

- *FM*: the selected baseline model (in this case FM)
- *FM-GCE*: baseline model using the GCE layer instead of usual embeddings
- *FM-GCE-SINFO*: baseline model using the GCE layer with side-information (represented as input features) for some nodes

We perform the comparisons on four benchmark datasets: MovieLens-100k [17], LastFM-1K [46], User-Book Interactions [38], [39] and Drug-Disease Interactions [30].

TABLE 1. Dataset statistics in terms of number of users, number of items and number of interactions among them.

dataset	#users	#items	#interactions
ML-100K	943	1,682	100,000
Books	3,909	1,412	172,124
LastFM	709	5,000	468,740
Drug-Disease	549	2,575	127,406

On the one hand, the first three datasets consist of user-item interactions on a diverse set of items: movies, songs and books, respectively. In addition, they all hold implicit context information that can be inferred from the data; for example, we can determine the order of the item(s) that the user interacted with by sorting all items of a given user according to the interaction's timestamp. On the other hand, the last dataset⁵ consists of interactions between drugs ("users") and diseases ("items"), originally used for drug-target interaction prediction and drug re-purposing [30]. The dataset includes protein interaction data for both drugs and diseases, which we use as context by taking the intersection of those which were present when a drug-disease interaction occurred. In addition, it also provides side-effect information that is associated to drugs, which we can use as side-information.

Furthermore, in our experiments, we focus on the top- K recommendation task based on implicit feedback data, which is a more common scenario compared with rating predictions. As a result, we transform the datasets that contain ratings to binary user-item interactions and, further, apply some pre-processing to them, as discussed in IV-B. We report the following statistics for the final versions of the datasets⁶:

A. EXPERIMENTAL SETUP

1) BASELINES

We implemented the following baseline models using Pytorch [24]. For each of these baselines, we tested the performance of the three variants as discussed before.⁷ Note that all baselines need to be, and thus are, context-aware i.e. they model user-item-context interactions. Moreover, all baselines are trained and tested on the same data, being tuned according to the same criteria and using the same loss function.

- **Matrix Factorization (MF)** [3]: This is the classic matrix factorization method. We extended it to tensor factorization (decomposition in n matrices instead of just two) in order to allow using context.
- **Factorization Machines (FM)** [31]: This is the original FM. It is a strong baseline that computes the pair-wise operation between features instead of computing the dot product across all dimensions.
- **Neural Collaborative Filtering (NCF)** [20]: This baseline resembles the original FM, but adds another term of concatenated input vectors that are fed into a MLP.

⁵ Available at <https://github.com/luoyunan/DTINet/>

⁶ Statistics reported after applying all data transformations needed.

⁷ Third variant will be done just if dataset provides side-information.

2) PARAMETER SETTINGS

To ensure a fair comparison of the models' performance, we train all of them by optimizing the BPR loss [32] with the Adam optimizer [25]. We use Bayesian Optimization [8] strategy in order to tune the hyperparameters and follow the same criteria in all cases. Thus, we determine the best hyperparameters for each of the models by tuning the learning rate on the range [0.0001, 0.0005, 0.001, 0.005, 0.01]; batch size on the range [256, 512, 1024, 2048], and dropout on range [0, 0.15, 0.5]. The embedding size is set to 64 for all models.⁸ We run all the experiments for a maximum of 150 epochs and perform early stopping when the evaluation metrics defined in subsection IV-A3 stops improving for more than 5 consecutive epochs.

3) EVALUATION PROTOCOL

The evaluation of all models follows the standard offline top- K evaluation, where the target is to generate a ranked list (according to the predicted scores) of K items that a user is most likely to interact with. We use the *leave-one-out* strategy, which has been widely adopted in literature [19], [20], [45], for splitting the dataset to train, validation and test sets. We held out the last two interactions of the users for the validation and test set, if the timestamp information is available. We, thus, require users to have interacted with at least three items so that we can -at a minimum- allocate one observation for training, one for validation and one for testing. The datasets have been pre-processed based on this constraint (we provide a more detailed description of the pre-processing in subsection IV-B).

We report the Hit Ratio (HR) and Normalized Discounted Cumulative Gain (NDCG) metrics on the test set by ranking among all items for evaluating the recommendation quality of the models:

- **HR@K:** A recall-based metric, measuring whether the test item is in the top- K positions of the recommendation list.
- **NDCG@K:** It is a rank-sensitive measure of quality, which provides information on the ranking position of the ground truth sample; higher scores are assigned to the top-ranked items and can be regarded as a weighted version of HR@K.

When evaluating the performance, the models give predicted scores (logits) of each item for a given user and context variables in the validation or test set. All items are sorted in the decreasing order of their scores and top- K items are returned as recommendation list. If the ground truth item is presented in the recommendation list, HR would be 1 for this user, otherwise, it would be 0. The final HR is reported after averaging for all users and an analogous calculation is done for NDCG. All experiments are conducted with 10 different random seeds and the average scores are reported.

⁸In this paper, we report the results when using one GCE layer because it will not introduce additional trainable parameters compared with the commonly used embedding table.

TABLE 2. Performance comparison for all baselines and their GCE variants, without and with movie's genre as item's side-information, for the ML-100k dataset. Sig. levels (two tails, corrected for mult. comparisons): * $p < .01$; ** $p < .001$.

ML-100k [last-clicked-item]				
	Top-10		Top-20	
	HR	NDCG	HR	NDCG
MF	0.135	0.071	0.210	0.089
MF-GCE	0.222**	0.119**	0.329**	0.145**
MF-GCE-SI	0.214**	0.114**	0.327**	0.143**
FM	0.186	0.099	0.286	0.124
FM-GCE	0.220**	0.115**	0.329**	0.145**
FM-GCE-SI	0.222**	0.118**	0.327**	0.144**
NCF	0.186	0.100	0.279	0.124
NCF-GCE	0.212**	0.113**	0.305**	0.136**
NCF-GCE-SI	0.211**	0.112**	0.310**	0.137**

Finally, to investigate further the performance differences across the different model variants, we run Friedman's ANOVA as an omnibus test and, if the result is statistically significant, we use the Wilcoxon signed-rank test for pairwise comparisons (for which we correct the level of significance to control the false discovery rate by using the Benjamini-Hochberg correction [5], [6]). Both Friedman and Wilcoxon tests are non-parametric, which means they do not make any assumptions about the data distribution and are therefore appropriate for our experimental setup [12]. We highlight the statistically significant differences between the baselines and their -GCN and -SINFO variants in Tables 2-4 and 6.

B. PERFORMANCE

We give a brief description of each dataset and the respective context, as well as a description of the pre-processing steps that we applied. We train the standard baseline model, as well as its corresponding GCE variant, without and with side-information (if data provides it), for every dataset and baseline. Please, note that the three versions of each model are trained end-to-end by using context information into account (if using GCE in a N -partite graph).

1) ML-100k DATASET

The MovieLens 100k dataset⁹ contains 100,000 ratings from 943 users on 1,682 movies. The ratings are provided in the form of $\langle \text{userID}, \text{itemID}, \text{rating}, \text{timestamp} \rangle$ tuples and each user has a minimum of 20 ratings. For this dataset, we used the last interacted movie of each user as the context. The context is the previous movie that a user selected before the current item interaction.

As they provide the genre for each of the movies, they are used as side-information to train each baseline one additional step when applying the GCE layer in the *BASELINE-GCE-SI* experiment.¹⁰

⁹<https://grouplens.org/datasets/movielens>

¹⁰Note that *SI* refers to Side-Information.

As mentioned earlier, we converted the ratings to binary data to use them as implicit feedback. This is done by taking all interactions as positive samples and performed negative sampling in every update step to generate non-interaction samples. No other pre-processing is applied.

Table 2 shows the results of the selected baselines when generating top-10 and top-20 recommendation. We observe that all models performed significantly better when using GCE and that the best results are achieved by the GCE layer, when also adding side-information. On the other hand, adding side-information in the MF-GCE does not introduce any improvement. However, it should be noted that MF-GCE results achieve either the same or the best performance among the nine (9) model variants on this data. This might strengthen the statements from [34], which claims that with a proper hyperparameter selection, a simple dot product substantially outperforms more expressive models.

2) LastFM-1k DATASET

The LastFM-1K¹¹ is a subset of LastFM¹² which offers a user full listening history containing the whole listening habits (till May, 5th 2009) for nearly 1,000 users. This dataset contains $\langle \text{user}, \text{timestamp}, \text{artist}, \text{song} \rangle$ tuples collected from Last.fm API. In our experiments, we used the records of the last year (2009). Furthermore, we select those users with at least 3 interactions and the 5,000 most frequent songs. We used the timestamp to select the last song that the user interacted with, before each listening event, as the context, and the artist as side-information of the user.

Table 3 shows the results of the selected methods, without and with the GCE layer, in two different scenarios: not using or using artists as item's side-information. Here, we observe clearly how the GCE layer consistently improves the results for this data as well. We further note that adding side-information does not lead to better results when the model is expressive enough. This might be because side-information does not introduce any new features that allow the model to learn better representations and, thus, using artist as side-information may not be helpful on the addressed use case.

3) USER-BOOKS INTERACTION DATASET

The User-Books Interaction dataset¹³ is a set of user reading history. The extended version of this dataset (aprox. 64GB of size) includes, in addition, the associated timestamps. In our experiments, we randomly sampled approximately 1M interactions from 6, 000 different users and, in addition, ranked the items by their frequency to acquire the top 5, 000 read books. Then, we filter the users and force them to have, at least, 20 interactions per user, as done in the Movielens dataset. The final dataset included 172,124 interactions from 3,909 users on 1,412 books. The context used for this dataset is the last

TABLE 3. Performance comparison for all baselines and their GCE variants, without and with song's artist as item's side-information, for the LastFM dataset. Sig. levels (two tails, corrected for mult. comparisons): * $p < .01$; ** $p < .001$.

LastFM [last-clicked-item]				
	Top-10		Top-20	
	HR	NDCG	HR	NDCG
MF	0.328	0.258	0.365	0.269
MF-GCE	0.434**	0.331**	0.495**	0.342**
MF-GCE-SI	0.435**	0.333**	0.492**	0.347**
FM	0.458	0.358	0.513	0.367
FM-GCE	0.472*	0.368	0.524*	0.380**
FM-GCE-SI	0.471*	0.365	0.523*	0.378*
NCF	0.448	0.350	0.493	0.360
NCF-GCE	0.508**	0.424**	0.553**	0.434**
NCF-GCE-SI	0.489*	0.416**	0.540**	0.428**

TABLE 4. Performance comparison for all baselines and their GCE variants, with $k = 1$ (default) and $k = 2$ for the User-Books Interaction dataset. Sig. levels (two tails, corrected for mult. comparisons): * $p < .01$; ** $p < .001$.

Books [last-clicked-item]				
	Top-10		Top-20	
	HR	NDCG	HR	NDCG
MF	0.061	0.039	0.085	0.045
MF-GCE	0.155**	0.093**	0.217**	0.109**
FM	0.140	0.089	0.192	0.101
FM-GCE	0.160**	0.099**	0.214**	0.113**
NCF	0.138	0.088	0.192	0.102
NCF-GCE	0.150**	0.090	0.213**	0.106*

book that the user read before the current interaction and it does not provide any side-information.

Table 4 summarises the results of the experiments on the User-Books Interaction dataset. For all methods, we observe significant improvements when introducing the GCE layers, but, similarly to previous results, the improvement on the MF is more pronounced for being the one capturing less high-order interaction information.

4) DRUG-DISEASE INTERACTION DATASET

The Drug-Disease Interaction dataset¹⁴ is available for drug-target interaction prediction and drug re-purposing [30]. In this set of experiments, we treat the drug-disease associations as user-item relationships and try to predict new disease targets for the drugs to be re-purposed. For both drugs and diseases, for which their respective protein associations are provided, we compute the intersection of proteins on a drug-disease interaction and use it as context variable. Hence, we construct the 3-partite drug-disease-protein graph. Given a drug and a set of associated proteins, we try to predict a ranking of diseases for which the drug can be useful. Additionally, we use the side-effects provided for drugs as side-information on the last experiment to see whether it can increase the performance of a model in this case.

¹¹<http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>

¹²<https://www.last.fm>

¹³<https://sites.google.com/eng.ucsd.edu/ucsdbookgraph/shelves>

¹⁴<https://github.com/luoyunan/DTINet/tree/master/data>

TABLE 5. Recommendation Performance in NDCG@10 for FM baseline on ML-100k and User-Books Interaction dataset after removing the Top-k popular Items (a) or after removing the Top-k popular Users (b) in the Test Stage.

k	(a) w/o Top-k popular Items						(b) w/o Top-k popular Users					
	ML-100K			User-Books			ML-100K			User-Books		
	FM	FM-GCE	Improv.	FM	FM-GCE	Improv.	FM	FM-GCE	Improv.	FM	FM-GCE	Improv.
0	0.0726	0.0815	12.25%	0.0567	0.0665	17.28%	0.0726	0.0815	12.25%	0.0567	0.0665	17.28%
10	0.0067	0.0078	16.41%	0.0045	0.0052	15.55%	0.0731	0.0789	10.79%	0.0572	0.0667	16.61%
25	0.0063	0.0074	17.46%	0.0039	0.0049	25.64%	0.0774	0.0863	11.49%	0.0594	0.0683	14.99%
50	0.0053	0.0072	35.84%	0.0035	0.0040	14.28%	0.0859	0.0911	6.05%	0.0600	0.0689	14.83%

TABLE 6. Performance comparison for all baselines and their GCE variants, without and with side-effects as drug's side-information, for the Drug-Disease dataset. Sig. levels (two tails, corrected for mult. comparisons): * $p < .01$; ** $p < .001$.

Drug-Disease [proteins-intersection]				
	Top-10		Top-20	
	HR	NDCG	HR	NDCG
MF	0.186	0.119	0.251	0.135
MF-GCE	0.270**	0.166**	0.354**	0.185**
MF-GCE-SI	0.263**	0.160**	0.351**	0.182**
FM	0.307	0.196	0.399	0.218
FM-GCE	0.345**	0.225**	0.453**	0.252**
FM-GCE-SI	0.356**	0.227**	0.450**	0.251**
NCF	0.309	0.200	0.398	0.221
NCF-GCE	0.364**	0.236**	0.451**	0.256**
NCF-GCE-SI	0.362**	0.232**	0.452**	0.255**

As we can observe in Table 6, for the two most expressive models (FM, NCF), the more data we incorporate the more improvement we get. This is because, in this scenario, data fits perfectly a graph-structure. Therefore, adding the proteins intersection of a given drug and disease helps by itself the learning of the model. Additionally, incorporating this information in a graph-based structure leads to further improvements. Furthermore, the side information we present here is actually pharmacological or phenotypic information (side-effects) that is specific and directly related with each of the drugs it is associated with. In practice, this indicates that the properties of the GCE layer achieve the most improvement when it is based on data with implicit graph-shape structure.

In Figure 4 we show the performance in terms of NDCG@10 and NDCG@20 of FM baseline in four possible scenarios:

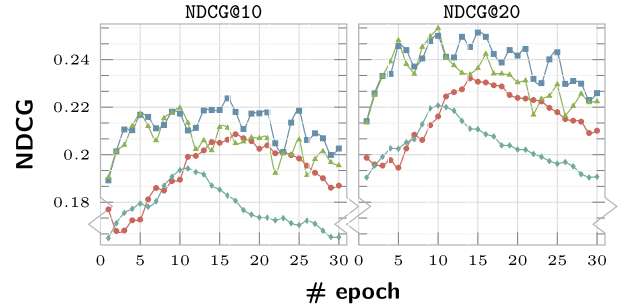
- 1) Drug-disease interactions using FM baseline.¹⁵
- 2) Drug-proteins-disease,¹⁶ using FM baseline.
- 3) Drug-proteins-disease, using GCE layer with FM baseline (FM-GCE).
- 4) Drug-proteins-disease with side-effects as feature vectors of drug nodes, using GCE layer with FM baseline and side-information (FM-GCE-SI).

C. ANALYSIS

In our experiments, we examined the GCE in MF, FM and NFM. In all cases, the performance of the models improves

¹⁵It is analogous to a non-context scenario, like user-item problems.

¹⁶Using protein-intersection as context, as described previously.

**FIGURE 4.** Comparison for FM baseline in Drug-Disease Interaction dataset: drug-disease interaction with FM (♦), drug-disease-protein interaction with FM (●), drug-disease-protein interaction with FM-GCE (▲), and drug-disease-protein interaction with FM-GCE-SI, using side-effect as drug's features (■).

when the GCE layers are used. This demonstrates that introducing GCE layers leads to statistically significant improvements with respect to the corresponding baseline models. In what follows, we further analyze the performance on long-tail users and items to see whether the GCE layer can preserve it's benefits on long tail users or items [47]. We perform the experiments in terms of NDCG@10 metric for FM baseline, and FM-GCE model on two datasets: ML-100K and User-Books interaction.

1) LONG-TAIL ITEMS

As shown in Table 5(a), for both without and with GCE layer, performance on NDCG@10 drops a lot when subtracting the K popular items. This might be because popular items dominate the top of recommendation lists and also due to the little amount of relevant neighbours that those items may have. However, the proposed GCE layer achieves more significant improvements when recommending less popular items - the improvement increases the more popular items we subtract.

2) LONG-TAIL USERS

Similarly, as shown in Table 5(b), we sort users by the total of interactions they had and report the performance results when removing the k-popular users. We observe a trend that the relative improvement of the baseline methods when using the GCE layers is larger when removing popular items. This shows the potential of the GCE layer to improve the performance of baseline models in settings where less information is available, by propagating information over the

TABLE 7. First-epoch recommendation performance in terms of NDCG@10 metric for all baseline models in all datasets.

	ML-100K	Books	LastFM	Drugs-Disease
MF	0.0302	0.0189	0.1594	0.0320
MF-GCE	0.0647	0.0369	0.2150	0.1594
FM	0.0719	0.0410	0.2775	0.1649
FM-GCE	0.0872	0.0548	0.3075	0.1905
NCF	0.0741	0.0126	0.2600	0.1795
NCF-GCE	0.0877	0.0666	0.2701	0.1857

N-partite graph. On the user side, we observe less of this effect and note that GCE maintains its relative performance with respect to the baseline models in most settings.

D. INDUCTIVE BIAS EFFECT

Inductive bias are the assumptions encoded in a model to generalize its performance. In the case of a Graph Convolutional Embeddings, the inductive bias is called **relational** and it is based on the assumption that elements that are connected in the graph must have similar representations. This inductive bias in GCE is essentially encoded by the adjacency matrix of the model.

Table 7 shows the model performance in ML-100k and User-Books Interaction datasets¹⁷ for each of the baselines, after only one iteration of the training process with a random mini-batch of data. This experiment is designed to show the effect of GCE's inductive bias with regard to performance. We observe that the GCE layer allows the model to achieve significant better results than baseline models when evaluated after one training step, thus leading to consistently better performance. The GCE layer by definition encodes the graph topology (i.e. user-item-context interaction) information into the node embedding vectors, while conventional embeddings are initialized randomly. In the latter case, gradient updates are the only source of information about interaction patterns, as well as more complex information like the nodes' intrinsic features. Embeddings from the proposed GCE layer represent the interaction patterns explicitly, hence the training process is decoupled and the model can learn more complex signals.

We also observe that the dataset structure needs to be consistent in order to apply the GCE layer successfully. For instance, if the dataset structure concentrates the majority of the interactions on some specific users, or if the majority of items are clicked by just one user, that can lead to no improvement when applying the GCE layer. In those situations, some users seen as outliers can be removed or top-frequent items can be selected, respectively.

E. DISCUSSION

FM have been the reference models in the area of context-aware recommendation models. However, they do not model effectively high-order and nonlinear interaction

signals. Several recent efforts focused on enriching FM with neural networks, such as NFM [18] and NCF [20] models, which are considered strong baselines.

Additionally, Rendle *et al.* [34] showed that, by carefully optimizing the training process, expressive models such as NCF were not needed to outperform the classical MF. However, in our case, where context dimensions are introduced, we do not observe this pattern. This can be due to the fact that MF is extended to tensor factorization in order to be adapted for context-aware recommendation. More specifically, we observe that it is more efficient to compute pair-wise operations (like FM does) and one of the reasons might be that the number of parameters becomes too high when computing inner product along more than 2 dimensions. What can be observed in any of the datasets is that FM performance is much closer in terms of performance to NCF results, hence this might corroborate the idea of [34], but for more than just user-item dimensions.

Additionally, it has to be noted that applying the GCE layer on MF leads to more notable improvements, as it helps to capture high-order interactions between the various dimensions of the context and user-item interactions as effectively as in other DL-based models. This further demonstrates that the proposed GCE layer increases even more the performance of downstream recommendation models that are not so strong, acting as a potential equalizer of performance among recommendation models.

V. CONCLUSION

The main goal of this work was to ease the use of GCNs in the area of RS by building a GCN model that can be used as an embedding layer in practically any downstream recommendation model, by processing generalized user-item-context interactions. In this paper, we achieved two contributions: 1) we generalized GCNs in *N*-partite interaction graphs to take context variables into account and 2) we integrated the generated embeddings in downstream recommendation models trained in an end-to-end fashion.

We presented an effective model for capturing high-order and non-linear interaction signals in context-aware recommender systems, which we call Graph Convolutional Embeddings. This architecture is based on the message passing principle of GCN, it is scalable and can be used with any deep learning recommendation model that naturally allows to incorporate context, such as FM, NCF, DFM, etc. Using our layer allows to easily incorporate side-information for each of the nodes.

From the experimental results, we observed that using graph convolutions helps to capture patterns in the contextual data and lead to better model performance, especially in relatively large and sparse datasets. We argue that this is due to the fact that graph convolutions don't only encode in each node embedding the information of the node itself, but also the information from correlated nodes, thus, leverage the graph topology and connectivity information contained by the constructed interaction graph.

¹⁷The results in other datasets are similar and are omitted due to the space limitation.

Code is publicly available at <https://github.com/paulagd/GCE>.

ACKNOWLEDGMENT

Part of this work was carried out during an internship of P.G.D. in Telefonica Research, Barcelona, Spain. The GPUs used in this research are provided by NVIDIA Corporation with 1080Ti Series.

REFERENCES

- [1] G. Adomavicius and A. Tuzhilin, "Context-aware recommender systems," in *Recommender Systems Handbook*. Berlin, Germany: Springer, 2011, pp. 217–253.
- [2] G. Alexandridis, G. Siolas, and A. Stafylopatis, "Enhancing social collaborative filtering through the application of non-negative matrix factorization and exponential random graph models," *Data Mining Knowl. Discovery*, vol. 31, no. 4, pp. 1031–1059, Jul. 2017.
- [3] L. Baltrunas, B. Ludwig, and F. Ricci, "Matrix factorization techniques for context aware recommendation," in *Proc. 5th ACM Conf. Recommender Syst. (RecSys)*, 2011, pp. 301–304.
- [4] O. Barkan and N. Koenigstein, "Item2Vec: Neural item embedding for collaborative filtering," in *Proc. IEEE 26th Int. Workshop Mach. Learn. Signal Process. (MLSP)*, Sep. 2016, pp. 1–6.
- [5] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. Roy. Stat. Soc. B, Methodol.*, vol. 57, no. 1, pp. 289–300, Jan. 1995.
- [6] Y. Benjamini and Y. Hochberg, "On the adaptive control of the false discovery rate in multiple testing with independent statistics," *J. Educ. Behav. Statist.*, vol. 25, no. 1, pp. 60–83, 2000.
- [7] R. van den Berg, T. N. Kipf, and M. Welling, "Graph convolutional matrix completion," 2017, *arXiv:1706.02263*. [Online]. Available: <http://arxiv.org/abs/1706.02263>
- [8] J. Bergstra, B. Komer, C. Eliasmith, D. Yamins, and D. D. Cox, "Hyperopt: A Python library for model selection and hyperparameter optimization," *Comput. Sci. Discovery*, vol. 8, no. 1, Jul. 2015, Art. no. 014008.
- [9] J. B. Estrach, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and deep locally connected networks on graphs," in *Proc. 2nd Int. Conf. Learn. Represent. (ICLR)*, vol. 2014, 2014.
- [10] J. Chen, T. Ma, and C. Xiao, "FastGCN: Fast learning with graph convolutional networks via importance sampling," 2018, *arXiv:1801.10247*. [Online]. Available: <https://arxiv.org/abs/1801.10247>
- [11] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, Red Hook, NY, USA, 2016, pp. 3844–3852.
- [12] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, Jan. 2006.
- [13] D. K. Duvenaud, D. Maclaurin, J. Iparraguirre, R. Bombarell, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional networks on graphs for learning molecular fingerprints," in *Proc. Adv. Neural Inf. Process. Syst.*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, 2015, pp. 2224–2232.
- [14] W. Fan, Y. Ma, Q. Li, Y. He, E. Zhao, J. Tang, and D. Yin, "Graph neural networks for social recommendation," in *Proc. World Wide Web Conf. (WWW)*, 2019, pp. 417–426.
- [15] H. Guo, R. Tang, Y. Ye, Z. Li, and X. He, "DeepFM: A factorization-machine based neural network for CTR prediction," 2017, *arXiv:1703.04247*. [Online]. Available: <http://arxiv.org/abs/1703.04247>
- [16] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1024–1034.
- [17] F. M. Harper and J. A. Konstan, "The MovieLens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 1–19, Dec. 2015.
- [18] X. He and T.-S. Chua, "Neural factorization machines for sparse predictive analytics," in *Proc. 40th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Aug. 2017, pp. 355–364.
- [19] X. He, Z. He, J. Song, Z. Liu, Y.-G. Jiang, and T.-S. Chua, "NAIS: Neural attentive item similarity model for recommendation," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 12, pp. 2354–2366, Dec. 2018.
- [20] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 173–182.
- [21] M. Henaff, J. Bruna, and Y. LeCun, "Deep convolutional networks on graph-structured data," 2015, *arXiv:1506.05163*. [Online]. Available: <http://arxiv.org/abs/1506.05163>
- [22] M. Iqbal, M. A. Ghazanfar, A. Sattar, M. Maqsood, S. Khan, I. Mehmood, and S. W. Baik, "Kernel context recommender system (KCR): A scalable context-aware recommender system algorithm," *IEEE Access*, vol. 7, pp. 24719–24737, 2019.
- [23] A. Karatzoglou, X. Amatriain, L. Baltrunas, and N. Oliver, "Multiverse recommendation: N-dimensional tensor factorization for context-aware collaborative filtering," in *Proc. 4th ACM Conf. Recommender Syst.*, 2010, pp. 79–86.
- [24] N. Ketkar, "Introduction to Pytorch," in *Deep Learning With Python*. Berlin, Germany: Springer, 2017, pp. 195–208.
- [25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [26] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [27] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 8, pp. 30–37, Aug. 2009.
- [28] J. Li, P. Feng, and J. Lv, "ICAMF: Improved context-aware matrix factorization for collaborative filtering," in *Proc. IEEE 25th Int. Conf. Tools Artif. Intell.*, Nov. 2013, pp. 63–70.
- [29] J. Lian, X. Zhou, F. Zhang, Z. Chen, X. Xie, and G. Sun, "XDeepFM: Combining explicit and implicit feature interactions for recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 1754–1763.
- [30] Y. Luo, X. Zhao, J. Zhou, J. Yang, Y. Zhang, W. Kuang, J. Peng, L. Chen, and J. Zeng, "A network integration approach for drug-target interaction prediction and computational drug repositioning from heterogeneous information," *Nature Commun.*, vol. 8, no. 1, pp. 1–13, 2017.
- [31] S. Rendle, "Factorization machines," in *Proc. IEEE Int. Conf. Data Mining*, Dec. 2010, pp. 995–1000.
- [32] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," 2012, *arXiv:1205.2618*. [Online]. Available: <http://arxiv.org/abs/1205.2618>
- [33] S. Rendle, Z. Gantner, C. Freudenthaler, and L. Schmidt-Thieme, "Fast context-aware recommendations with factorization machines," in *Proc. 34th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2011, pp. 635–644.
- [34] S. Rendle, W. Krichene, L. Zhang, and J. Anderson, "Neural collaborative filtering vs. matrix factorization revisited," in *Proc. 14th ACM Conf. Recommender Syst.*, Sep. 2020, pp. 240–248.
- [35] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia, "Learning to simulate complex physics with graph networks," 2020, *arXiv:2002.09405*. [Online]. Available: <http://arxiv.org/abs/2002.09405>
- [36] J. Sun, Y. Zhang, C. Ma, M. Coates, H. Guo, R. Tang, and X. He, "Multi-graph convolution collaborative filtering," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2019, pp. 1306–1311.
- [37] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*. [Online]. Available: <http://arxiv.org/abs/1710.10903>
- [38] M. Wan and J. McAuley, "Item recommendation on monotonic behavior chains," in *Proc. 12th ACM Conf. Recommender Syst.*, Sep. 2018, pp. 86–94.
- [39] M. Wan, R. Misra, N. Nakashole, and J. McAuley, "Fine-grained spoiler detection from large-scale review corpora," 2019, *arXiv:1905.13416*. [Online]. Available: <http://arxiv.org/abs/1905.13416>
- [40] X. Wang, X. He, Y. Cao, M. Liu, and T.-S. Chua, "KGAT: Knowledge graph attention network for recommendation," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 950–958.
- [41] X. Wang, X. He, M. Wang, F. Feng, and T.-S. Chua, "Neural graph collaborative filtering," in *Proc. 42nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2019, pp. 165–174.
- [42] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2019, pp. 6861–6871.
- [43] X. Xin, B. Chen, X. He, D. Wang, Y. Ding, and J. Jose, "CFM: Convolutional factorization machines for context-aware recommendation," in *Proc. 28th Int. Joint Conf. Artif. Intell.* Palo Alto, CA, USA: AAAI Press, Aug. 2019, pp. 3926–3932.

- [44] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2018, pp. 974–983.
- [45] F. Yuan, G. Guo, J. M. Jose, L. Chen, H. Yu, and W. Zhang, "LambdaFM: Learning optimal ranking with factorization machines using lambda surrogates," in *Proc. 25th ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2016, pp. 227–236.
- [46] E. Zangerle, M. Pichl, W. Gassler, and G. Specht, "#nowplaying music dataset: Extracting listening behavior from Twitter," in *Proc. 1st Int. Workshop Internet-Scale Multimedia Manage. (WISMM)*, 2014, pp. 21–26.
- [47] Y. Zhang, F. Sun, X. Yang, C. Xu, W. Ou, and Y. Zhang, "Graph-based regularization on embedding layers for recommendation," *ACM Trans. Inf. Syst.*, vol. 39, no. 1, pp. 1–27, Jan. 2021.



PAULA G. DURAN received the M.Sc. degree in advanced telecommunication technologies from Universitat Politècnica de Catalunya (UPC). She is currently pursuing the Ph.D. degree in mathematics and computer science with Universitat de Barcelona (UB), in the area of recommender systems. Her research interests include machine learning for recommendation, including deep learning, graph learning, and computer vision techniques.



ALEXANDROS KARATZOGLOU received the Ph.D. degree in machine learning from the Vienna University of Technology, while also being a frequent visitor with the Statistical Machine Learning Group, NICTA, Canberra, ACT, Australia. He worked as a Research Scientist with the Telefónica's Innovation Unit, Telefonica I+D, Barcelona, Spain, where he conducted research on machine learning for recommender systems, and later on became the Director of the Research

Group. He taught courses on deep learning and computational machine learning at the Graduate School of Economics Masters Course in Data Science, Barcelona, and at the GSE Data Science Summer School. He is currently a Staff Research Scientist with Google Research, London. He is also the author of kernlab, a fairly popular open source machine learning package for R.



JORDI VITRIÀ joined Universitat de Barcelona (UB), in 2007, where he is currently a Full Professor, and where he teaches an introductory course on algorithms and advanced courses on data science and deep learning. From April 2011 to January 2016, he worked as the Head of the UB's Applied Mathematics and Analysis Department. He is also a member of the Mathematics and Computer Science Department, UB. His research, when personal computers had 128 KB of memory, was originally oriented towards digital image analysis and how to extract quantitative information from them, but soon evolved towards computer vision problems. After a postdoctoral year at the University of California at Berkeley, in 1993, he focused on Bayesian methods for computer vision methods. He is leading a research group working in deep learning, computer vision, and machine learning. He has authored more than 100 peer-reviewed articles and holds eight international patents. He has directed 14 Ph.D. theses in the area of machine learning and computer vision. He has been the leader of a large number of research projects at international and national level.



XIN XIN received the M.Sc. degree in software engineering from Shanghai Jiaotong University, and the Ph.D. degree in computing science from the University of Glasgow, in the area of recommender system. His research interests include machine learning for recommendation, including deep learning, graph learning, and reinforcement learning.



IOANNIS ARAPAKIS received the M.Sc. degree in information technology from the Royal Institute of Technology (KTH), Sweden, and the Ph.D. degree from the SoCS, University of Glasgow, in the area of information retrieval.

From 2011 to 2015, he worked as a Research Scientist with Yahoo Labs, where he conducted research in the areas of data mining, IR, and HCI. During that time, he was a member of the User Engagement, Web Retrieval, and Ad Processing and Retrieval groups. He is currently a Research Scientist with the Telefónica's Innovation Unit, Telefonica I+D, Barcelona, Spain, where he conducts cutting-edge research on customer behavior, human attention, mobile phone use, and mobile network optimization. His research interests include predictive modeling, pattern recognition, time series analysis, neuro-physiological signal processing, user engagement, multimedia mining and search, social media analysis, and sentiment extraction.

...